Università della Svizzera italiana

Faculty of Informatics

Operating Systems

PintOS: Report Project #4

Group #6
Davide Frova, Costanza Rodriguez Gavazzi

Fall Semester, April 24, 2024

1 Files changed

- **/pintos/threads/thread.h** (*modified*)
- **/pintos/threads/thread.c** (*modified*)
- **/pintos/userprog/process.c** (*modified*)
- **/pintos/userprog/syscall.c** (*modified*)
- **/pintos/lib/syscall-nr.h** (*modified*)

2 Changes

/pintos/threads/thread.h

- Added **struct thread * parent** and **int * exit_status** in **struct thread**
- Added method signature for the new method added in **/pintos/threads/thread.c**

/pintos/devices/thread.c

- (*added*) **Method**:
  - **struct thread * thread_get(tid_t tid)**, given a tid_t, returns the corresponding thread. This is achived by iterating over the list of all threads and returning the one with the same tid. If no thread is found, it returns **NULL**.
- (*modified*) **tid_t thread_create (const char *name, int priority, thread_func *function, void *aux)**: We added the following lines of code to the function:

  - **t->parent = NULL**: set the parent of the new thread to **NULL**.
  - **t->exit_status = -80085**: set the exit status of the new thread to -80085 as a default value. This value is chosen to be different from the default value of the exit status of the main thread.

/pintos/userprog/process.c

- (*added*) **#define MAX_ARGS 128**: We added a define for the maximum number of arguments that can be passed to a process. This is used to allocate the space for the arguments array in the **process_execute** and **start_process** functions.

- (*modified*) **tid_t process_execute (const char *file_name)**: In this function we added a parsing of the **file_name** to extract the arguments. The parsing is done by using the method **char *strtok_r (char *, const char *, char **)**, this method is used to split the string by the space character. After this we pass to the **thread_create** function the first token as the name of the file. The entire string is then passed to the **start_process** function as the argument via the **void *aux** parameter of the **thread_create** function.

- (*modified*) **static void start_process (void *file_name_)**: This function is used to push all the necessary parameters and needed data onto the stack.

  These are the elements that are pushed onto the stack using the **if_.esp** pointer

  - Parse the ***file_name** to extract all the arguments and file name, we use the same strategy as in the **process_execute** function.
  - Push the **arguments** in reverse order while saving the corresponding pointers
  - Compute and push the **word align**
  - Push the **argv[agc]** so the 0 value
  - Push the **pointers** of the arguments in reverse order
  - Push the **argv** pointer
  - Push the **argc** (argument count)
  - Push the **return address** with value 0

- (*modified*) **int process_wait(tid_t child_tid)**: Implemented the function to handle the wait of a process.

  First we get the corresponding **struct thread *child** by using the **thread_get** function defined in **/pintos/devices/thread.c**

  Then we perform some checks, we check if the **thread_get** function returned **NULL** or if the **child->parent** is not **NULL**. If those are the cases we return -1 as are not to be handled and some error has occurred.

  After this we assign to the **child** the parent thread corresponding to the **thread_current()**

  Then we assign the address for it's **exit_status**

  Finally we disable **interrupts**, block the current thread, and after that re-enable the interrupts and return the resulting exit status of the child.

  The thread will get **unblocked** after the child exits and terminates it's execution.

/pintos/userprog/syscall.c

- (*modified*) **static void syscall_handler (struct intr_frame *f UNUSED)** : In this function we added a switch statement to handle the different syscall codes.

  The syscall code is read from the stack and stored in the **enum syscall_code s_code** variable. This will be used as a switch case to handle the different syscalls.

  We added the following cases:

  - **SYS_WRITE**: To handle the write syscall we read from the stack:
    - **int fd**: the file descriptor
    - **char *buffer**: the buffer address
    - **unsigned int size**: the size of the buffer

    We then call the **void putbuf (const char *buffer, size_t n)** function to write the buffer to the console.
  - **SYS_EXIT**: To handle the exit syscall we read from the stack:
    - **int exit_status**: the exit status

    We then set the exit status of the current thread to the status, save it into **f->eax**, **print** the exit statement of the current thread, **unblock** the parent of the current thread and call the **thread_exit()** function.
  - **default**: We added a default case to handle the case in which the syscall code is not recognized. In that case, we print an error message and call the **thread_exit()** function.

/pintos/lib/syscall-nr.h

- We added the name **syscall_code** to the enum declaration to be able to use it in the **syscall.c** file.